

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 3 – Konsep Dasar Pemrograman Berorientasi Objek



NAMA: Ken Rayana Ilma Madani

NIM: 103132430008

KELAS: S1 SD-05-01

PROGRAM STUDI SAINS DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

1. **Pemrograman Berorientasi Objek atau Object Oriented Programming (OOP)**

merupakan metode pemrograman yang membangun sistem berdasarkan objek. Dalam OOP, data dan fungsi dikelompokkan ke dalam suatu objek sehingga setiap objek dapat memiliki atribut, perilaku, serta dapat berinteraksi dengan objek lainnya. Konsep OOP bertujuan untuk membuat program lebih terstruktur dan mudah dikembangkan.

2. **Abstraksi**

Abstraksi merupakan proses menentukan hal-hal yang penting dari suatu objek dan mengabaikan hal-hal yang tidak diperlukan. Dalam abstraksi, objek dapat dijelaskan berdasarkan atribut yang dimiliki dan perilaku yang dapat dilakukan. Sebagai contoh, objek manusia memiliki atribut seperti nama, tinggi, dan berat, serta memiliki perilaku seperti makan, minum, dan berjalan.

3. **Enkapsulasi**

Enkapsulasi merupakan proses menggabungkan data objek dengan method yang mengolah data tersebut serta menyembunyikan detail implementasi dari bagian lain yang tidak memiliki hak akses. Enkapsulasi dapat melindungi data internal objek dari penggunaan atau perubahan yang tidak sesuai. Salah satu penerapannya adalah penggunaan akses `private` pada data dan penyediaan method sebagai antarmuka untuk mengakses data tersebut.

Enkapsulasi juga mendukung modularitas, yaitu setiap objek dapat dikelola secara independen. Dengan demikian, perubahan pada bagian internal suatu objek tidak selalu memengaruhi bagian program lainnya.

4. **Inheritance**

Inheritance atau pewarisan merupakan proses membuat kelas baru atau subclass yang mewarisi karakteristik dari kelas yang sudah ada atau superclass. Kelas turunan dapat menggunakan atribut dan method yang diwariskan sekaligus menambahkan atribut atau method baru sesuai kebutuhan. Konsep ini membantu menghindari penulisan ulang kode yang sudah tersedia pada superclass.

5. **Polymorphism**

Polymorphism berarti banyak bentuk. Konsep ini memungkinkan penggunaan antarmuka yang sama untuk memberikan perintah kepada objek, tetapi setiap objek dapat memberikan proses atau hasil yang berbeda. Dalam Java, polymorphism dapat diterapkan melalui method overloading, constructor overloading, maupun method overriding.

6. **Message atau Komunikasi Antarobjek**

Dalam OOP, objek dapat berkomunikasi dan berinteraksi dengan objek lainnya melalui message. Message dapat berupa pemanggilan method yang digunakan untuk meminta suatu objek melakukan tindakan tertentu. Informasi tambahan yang dibutuhkan oleh method dapat diberikan melalui parameter.

7. **Kelas**

Kelas merupakan cetak biru atau rancangan dari sebuah objek. Sebuah kelas dapat digunakan untuk membuat banyak objek dengan karakteristik yang telah ditentukan. Kelas memiliki anggota yang terdiri atas atribut dan method. Atribut digunakan untuk menyimpan data atau identitas objek, sedangkan method digunakan untuk mendefinisikan perilaku atau proses yang dapat dilakukan oleh objek. Dalam Java, kelas dideklarasikan menggunakan keyword `'class'`, kemudian diikuti dengan nama kelas serta anggota kelas yang berupa data dan method.

8. Object

Object atau objek merupakan hasil instansiasi dari suatu kelas. Dengan kata lain, objek merupakan bentuk nyata dari rancangan yang telah didefinisikan dalam kelas. Sebuah kelas dapat menghasilkan beberapa objek, dan setiap objek dapat memiliki nilai atribut yang berbeda. Dalam Java, objek dapat dibuat menggunakan keyword `'new'`.

9. Field

Field merupakan atribut atau anggota kelas yang digunakan untuk menyimpan data. Berdasarkan keterkaitannya, field dapat dibedakan menjadi class field dan instance field. Class field menggunakan modifier `'static'` sehingga berkaitan dengan kelas, sedangkan instance field tidak menggunakan `'static'` dan berkaitan dengan masing-masing objek atau instance dari kelas.

Class field merupakan field yang berhubungan dengan kelas itu sendiri, sehingga nilainya dapat diakses melalui nama kelas. Sementara itu, setiap objek memiliki salinan instance field masing-masing sehingga nilai field tersebut dapat berbeda antarobjek.

10. Method

Method merupakan fungsi atau prosedur yang terdapat dalam suatu kelas dan digunakan untuk melakukan suatu proses atau mendefinisikan perilaku objek. Method membantu memodularisasi program dengan memisahkan tugas-tugas tertentu ke dalam bagian yang lebih terstruktur. Method dapat menerima parameter dan dapat mengembalikan nilai.

Method yang mengembalikan nilai menggunakan tipe data tertentu sebagai nilai kembalian dan menggunakan `'return'`. Sedangkan method yang tidak mengembalikan nilai menggunakan keyword `'void'`.

11. Passing Parameter

Parameter pada method dapat digunakan untuk mengirimkan data ketika method dipanggil. Modul menjelaskan dua cara pelewatan argumen, yaitu pass by value dan pass by reference. Pass by value digunakan pada tipe data primitif dan perubahan pada parameter di dalam method tidak mengubah variabel asal. Sementara itu, pada objek dan array, referensi terhadap data dapat digunakan sehingga perubahan terhadap objek dapat terlihat pada objek asal.

12. Keyword `'this'`

Keyword `'this'` merupakan referensi khusus dalam Java yang menunjuk pada objek yang sedang digunakan. Keyword ini dapat digunakan untuk membedakan atribut

objek dengan parameter method yang memiliki nama sama. Selain itu, `this` juga dapat digunakan untuk memanggil constructor lain dari dalam sebuah constructor.

Penggunaan `this` dapat dilihat ketika parameter method memiliki nama yang sama dengan atribut kelas. Dengan menggunakan `this`, program dapat secara jelas menunjukkan bahwa variabel yang dirujuk merupakan atribut dari objek yang sedang digunakan.

B. Guided

1. Buku.Java

```
package com.kenrayana.projectmodul3.guided;

public class Buku {
    private String pengarang;
    private String judul;

    private Buku() {
        this("Rumah Kita", "GoodBles");
        // this ini digunakan untuk memanggil konstruktor yang
        // menerima dua parameter
    }

    private Buku(String judul, String pengarang) {
        this.judul = judul;
        this.pengarang = pengarang;
    }

    private void cetakKeLayar() {
        System.out.println("Judul : " + judul + " Pengarang : "
            + pengarang);
    }

    public static void main(String[] args) {
        Buku a, b;
        a = new Buku("Jurassic Park", "Michael Chricton");
        b = new Buku();
        a.cetakKeLayar();
        b.cetakKeLayar();
    }
}
```

Output :

```

--- exec:3.6.3:exec (default-cli) @ ProjectModul3 ---
Judul : Jurassic Park Pengarang : Michael Chricton
Judul : Rumah Kita Pengarang : GoodBles
-----
BUILD SUCCESS
-----
Total time: 0.873 s
Finished at: 2026-10-05T11:55:23+07:00
-----

```

2. Circle.Java

```

package com.kenrayana.projectmodul3.guided;
public class Circle {
    public static final double PI = 3.14159;

    public static double radiansToDegrees(double rads) {
        return rads * 180 / PI;
    }

    public double r;

    public double area() {
        return PI * r * r;
    }

    public double circumference() {
        return 2 * PI * r;
    }

    // Tambahkan main method di bawah ini
    public static void main(String[] args) {
        // Menguji static method
        System.out.println("1 Radian ke Derajat: " + Circle.radiansToDegrees(1));

        // Membuat objek dari class Circle
        Circle c = new Circle();
        c.r = 7; // Menentukan jari-jari

        // Menampilkan hasil perhitungan area & circumference
        System.out.println("Luas lingkaran: " + c.area());
        System.out.println("Keliling lingkaran: " + c.circumference());
    }
}

```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul3 ---
1 Radian ke Derajat: 57.295827908797776
Luas lingkaran: 153.93791
Keliling lingkaran: 43.98226
-----
BUILD SUCCESS
-----
Total time: 0.859 s
Finished at: 2026-10-05T11:54:20+07:00
-----
```

3. DemoLagu.Java

```
package com.kenrayana.projectmodul3.guided;

class Lagu {

    private String pencipta;
    private String judul;

    public void IsiParam(String judul, String pencipta) {
        this.judul = judul;
        this.pencipta = pencipta;
    }

    public void cetakKeLayar() {
        if (judul == null && pencipta == null) {
            return;
        }
        System.out.println("Judul : " + judul + ", pencipta : "
            + pencipta);
    }
}

class DemoLagu {

    public static void main(String[] args) {
        Lagu a = new Lagu();
        a.IsiParam("God Will Make A Way", "Don Moen ");
        a.cetakKeLayar();
    }
}
```

Output:

```

] --- exec:3.6.3:exec (default-cli) @ ProjectModul3 ---
] Judul : God Will Make A Way, pencipta : Don Moen
-----
BUILD SUCCESS
-----
Total time: 0.866 s
Finished at: 2026-10-05T11:53:17+07:00
-----

```

4. Method

```

//File TestPass.Java
package com.kenrayana.projectmodul3.guided;

class TestPass {

    int i, j;

    TestPass(int a, int b) {
        i = a;
        j = b;
    }
    //passed by value dengan parameter berupa tipe data primitif

    void calculate(int m, int n) {
        m = m * 10;
        n = n / 2;
    }
    //passed by reference dengan berupa tipe data class

    void calculate(TestPass e) {
        e.i = e.i * 10;
        e.j = e.j / 2;
    }
}

//file PassedByValue
package com.kenrayana.projectmodul3.guided;

class PassedByValue {

    public static void main(String[] args) {
        int x, y;
        TestPass z;
        z = new TestPass(50, 100);
        x = 10;
        y = 20;
        System.out.println("Nilai sebelum passed by value : ");
        System.out.println("x = " + x);
    }
}

```

```

        System.out.println("y = " + y);
    //passed by value
    z.calculate(x, y);
    System.out.println("Nilai sesudah passed by value : ");
    System.out.println("x = " + x);
    System.out.println("y = " + y);
    System.out.println("Nilai sebelum passed by reference : ");
    System.out.println("z.i = " + z.i);
    System.out.println("z.j = " + z.j);
    //passed by reference
    z.calculate(z);
    System.out.println("Nilai sesudah passed by reference : ");
    System.out.println("z.i = " + z.i);
    System.out.println("z.j = " + z.j);
    }
}

```

Output:

```

--- exec:3.6.3:exec (default-cli) @ ProjectModul3 ---
Nilai sebelum passed by value :
x = 10
y = 20
Nilai sesudah passed by value :
x = 10
y = 20
Nilai sebelum passed by reference :
z.i = 50
z.j = 100
Nilai sesudah passed by reference :
z.i = 500
z.j = 50

-----
BUILD SUCCESS
-----

Total time:  0.871 s
Finished at: 2026-10-05T11:51:43+07:00
-----

```

5. Abstraksi

```

//file abstraksi.java
package com.kenrayana.projectmodul3.guided.abstraksi;

public class Abstraksi {
    public static void main(String[] args){
        Kucing aren = new Kucing();

        aren.nama = "Aren Gentong";
        aren.ras = "Mujaer";
        aren.bersuara();
    }
}

```

```

}

//file kucing.Java
package com.kenrayana.projectmodul3.guided.abstraksi;

public class Kucing {
    String nama;
    String ras;

    void bersuara(){
        System.out.println(nama + ras + "meow");
    }
}

```

Output:

```

--- exec:3.6.3:exec (default-cli) @ ProjectModul3 ---
Aren GentongMujaermeow
-----
BUILD SUCCESS
-----
Total time: 0.843 s
Finished at: 2026-10-05T11:17:39+07:00
-----

```

6. Enkapsulasi

```

//file enkapsulasi.java
package com.kenrayana.projectmodul3.guided.enkapsulasi;

public class Enkapsulasi {
    public static void main(String[] args){
        Rekening rek = new Rekening();

        rek.tambahSaldo(20000000);
        rek.tampilkanSaldo();
    }
}

//file Rekening.Java
package com.kenrayana.projectmodul3.guided.enkapsulasi;

public class Rekening {
    private int saldo = 0;

    public void tambahSaldo(int jumlah){
        saldo = saldo + jumlah;
        System.out.println("Saldo berhasil ditambahkan");
    }
}

```

```

        public void tampilkanSaldo(){
            System.out.println("Saldo Anda: " + saldo);
        }
    }

```

Output:

```

--- compiler:3.15.0:compile (default-compile) @ ProjectModul3 ---
Nothing to compile - all classes are up to date.

--- exec:3.6.3:exec (default-cli) @ ProjectModul3 ---
Saldo berhasil ditambahkan
Saldo Anda: 20000000
-----
BUILD SUCCESS
-----
Total time: 0.860 s
Finished at: 2026-10-05T11:13:35+07:00
-----

```

C. Unguided

1. Soal Unguided 1

Sebuah tim mengumpulkan suhu harian suatu kota selama satu minggu (7 hari) dari sensor otomatis. Data ini sudah tersedia sebagai array di `main` sebelum diserahkan untuk diolah.

Pada salah satu hari, sensor gagal mengirim data. Hari yang gagal terekam ditandai dengan sebuah nilai penanda pada array, karena suhu asli tidak mungkin sebesar itu. Nilai penanda ini sama untuk semua data suhu yang diolah kapan pun dan oleh siapa pun — bukan sesuatu yang berbeda-beda tiap kali data dibuat — sehingga sebaiknya ditulis satu kali saja sebagai aturan tetap milik class, bukan diketik ulang sebagai angka `-1.0` di setiap method yang membutuhkannya.

Buatlah satu class yang bertugas mengolah data suhu ini — memeriksa apakah ada data kosong, mengisinya, dan menghasilkan statistik dasar dari data yang sudah bersih.

Jawaban :

```

package unguided;

public class PengolahSuhu {

    // Class field: milik class, dipakai bersama semua object, nilainya tidak bisa
    // diubah
    private static final double NILAI_KOSONG = -1.0;

    // Instance field: berbeda untuk setiap object
    private double[] suhuHarian;

    // Nama parameter sama dengan nama field, jadi 'this' dipakai secara eksplisit
    public PengolahSuhu(double[] suhuHarian) {
        this.suhuHarian = suhuHarian; // menyimpan REFERENSI, bukan salinan
    }
}

```

```

// Kebutuhan 3: tampilkan semua data, hari kosong ditandai
public void tampilkanData() {
    for (int i = 0; i < suhuHarian.length; i++) {
        if (suhuHarian[i] == NILAI_KOSONG) {
            System.out.println("Hari " + (i + 1) + " : (kosong)");
        } else {
            double bulat = Math.round(suhuHarian[i] * 10) / 10.0;
            System.out.println("Hari " + (i + 1) + " : " + bulat + "°C");
        }
    }
}

// Kebutuhan 4: cari index hari kosong, -1 jika tidak ada
public int cariIndexKosong() {
    for (int i = 0; i < suhuHarian.length; i++) {
        if (suhuHarian[i] == NILAI_KOSONG) {
            return i;
        }
    }
    return -1;
}

// Kebutuhan 5: isi hari kosong dengan rata-rata hari sebelum dan sesudahnya
public void isiDataKosong() {
    int i = cariIndexKosong();
    if (i != -1) {
        suhuHarian[i] = (suhuHarian[i - 1] + suhuHarian[i + 1]) / 2;
    }
}

// Kebutuhan 6: rata-rata suhu dari data yang sudah bersih
public double hitungRataRata() {
    double total = 0;
    for (double suhu : suhuHarian) {
        total += suhu;
    }
    return total / suhuHarian.length;
}

public static void main(String[] args) {
    double[] suhuHarian = { 30.4, 24.3, 26.8, -1.0, 31.4, 30.8, 32.9 };

    PengolahSuhu pengolah = new PengolahSuhu(suhuHarian);

    System.out.println("==== Data Suhu Awal ===");
    pengolah.tampilkanData();
    System.out.println();
    System.out.println("Index hari kosong (dimulai dari 0): " +
        pengolah.cariIndexKosong());
    System.out.println();

```

```

        pengolah.isiDataKosong();

        System.out.println("=== Data Suhu Setelah Pengisian ===");
        pengolah.tampilkanData();
        System.out.println();
        double rata = Math.round(pengolah.hitungRataRata() * 100) / 100.0;
        System.out.println("Rata-rata : " + rata + "°C");
        System.out.println();

        System.out.println("Isi array suhuHarian di main setelah isiDataKosong()
dijalankan:");
        System.out.print("[");
        for (int i = 0; i < suhuHarian.length; i++) {
            System.out.print(Math.round(suhuHarian[i] * 10) / 10.0);
            if (i < suhuHarian.length - 1) {
                System.out.print(", ");
            }
        }
        System.out.println("]");
        System.out.println("(ikut berubah: constructor menyimpan referensi array
yang sama)");

    }
}

```

Output :

```

--- exec:3.6.3:exec (default-cli) @ ProjectModul3 ---
=== Data Suhu Awal ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : (kosong)
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Index hari kosong (dimulai dari 0): 3

=== Data Suhu Setelah Pengisian ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : 29.1°C
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Rata-rata : 29.39°C

Isi array suhuHarian di main setelah isiDataKosong() dijalankan:
[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]
(ikut berubah: constructor menyimpan referensi array yang sama)
-----
BUILD SUCCESS
-----
Total time: 0.899 s
Finished at: 2026-10-05T11:05:36+07:00

```

Penjelasan:

1. Deklarasi Class

Program ini dibuat dalam satu class bernama PengolahSuhu dengan modifier public, sehingga class dapat diakses dari mana saja. Class ini berisi data dan method untuk mengolah suhu harian selama 7 hari. Method main juga diletakkan di dalam class ini sebagai titik awal eksekusi program.

2. Class Field NILAI_KOSONG

NILAI_KOSONG adalah class field bertipe double yang menyimpan nilai penanda data kosong, yaitu -1.0. Field ini dideklarasikan dengan kata kunci private static final. Kata kunci private membuat field hanya dapat diakses dari dalam class (enkapsulasi). Kata kunci static membuat field dimiliki oleh class, bukan oleh object tertentu, sehingga satu nilai dipakai bersama oleh semua object. Kata kunci final membuat nilainya tidak dapat diubah setelah ditetapkan, sehingga berperan sebagai konstanta. Dengan menuliskan -1.0 satu kali di sini, method lain cukup memanggil NILAI_KOSONG tanpa mengetik ulang angkanya.

3. Instance Field suhuHarian
suhuHarian adalah instance field bertipe array double yang menyimpan data suhu selama 7 hari. Setiap object PengolahSuhu memiliki suhuHarian sendiri. Field ini bersifat private sehingga tidak dapat diakses atau diubah langsung dari luar class.
4. Constructor
Constructor dijalankan secara otomatis saat object dibuat dengan kata kunci new. Constructor menerima satu parameter berupa array suhu yang namanya sama dengan nama field, yaitu suhuHarian. Karena namanya sama, kata kunci this dipakai secara eksplisit untuk membedakan keduanya: this.suhuHarian merujuk ke field milik object, sedangkan suhuHarian saja merujuk ke parameter. Constructor menyimpan array dari parameter ke dalam field. Array bertipe reference, sehingga yang disimpan adalah alamat array di memori, bukan salinan isinya. Konsekuensi dari hal ini dijelaskan pada bagian 10.
5. Method tampilkanData()
Method ini menampilkan seluruh data suhu (kebutuhan 3) dan tidak mengembalikan nilai (void). Perulangan for menelusuri array dari index 0 sampai elemen terakhir. Pada setiap iterasi, program memeriksa apakah nilai suhu sama dengan NILAI_KOSONG. Jika sama, dicetak tulisan "(kosong)" untuk menandai hari yang datanya hilang. Jika tidak sama, suhu dicetak bersama satuan °C. Index array dimulai dari 0, sedangkan penomoran hari dimulai dari 1, sehingga nomor hari yang dicetak adalah index ditambah 1. Sebelum dicetak, suhu dibulatkan ke satu angka di belakang koma menggunakan Math.round agar tampilannya rapi dan tidak muncul angka desimal yang panjang akibat sifat perhitungan bilangan double.
6. Method cariIndexKosong()
Method ini mencari hari yang datanya kosong (kebutuhan 4) dan mengembalikan nilai bertipe int. Array ditelusuri dari awal dengan perulangan for. Begitu ditemukan elemen yang bernilai NILAI_KOSONG, method langsung mengembalikan index elemen tersebut dengan return. Jika perulangan selesai tanpa menemukan data kosong, method mengembalikan -1 sebagai tanda bahwa tidak ada data yang kosong. Pada data soal, method ini mengembalikan 3, yaitu index untuk Hari 4.
7. Method isiDataKosong()
Method ini mengisi data yang kosong (kebutuhan 5) dan tidak mengembalikan nilai (void). Pertama, method memanggil cariIndexKosong() dan menyimpan hasilnya dalam sebuah variabel. Jika hasilnya bukan -1, artinya ada data kosong, maka elemen pada index tersebut diganti dengan rata-rata suhu hari sebelumnya (index dikurangi 1) dan hari sesudahnya (index ditambah 1). Teknik ini disebut imputasi, yaitu mengisi nilai yang hilang berdasarkan rata-rata data tetangga terdekatnya, dengan asumsi suhu tidak berubah drastis di antara dua hari tersebut.

Pada data soal, nilai yang diisikan adalah $(26.8 + 31.4) / 2 = 29.1$. Jika tidak ada data kosong, method tidak melakukan apa pun. Pemeriksaan ini juga mencegah error akibat pengaksesan array dengan index yang tidak valid.

8. Method `hitungRataRata()`

Method ini menghitung rata-rata suhu (kebutuhan 6) dan mengembalikan nilai bertipe `double`. Method menyiapkan variabel `total` yang bernilai awal 0, lalu menggunakan perulangan `for-each` untuk menambahkan setiap suhu di array ke `total`. Hasil penjumlahan kemudian dibagi dengan banyaknya elemen array (`panjang array`) dan dikembalikan sebagai hasil. Method ini sebaiknya dipanggil setelah data kosong diisi, karena nilai `-1.0` yang masih tersisa akan membuat hasil rata-rata tidak akurat.

9. Method `main`

Method `main` dimulai dengan menyiapkan array `suhuHarian` berisi 7 data suhu, dengan nilai `-1.0` pada hari ke-4 sebagai penanda data kosong. Selanjutnya dibuat sebuah object `PengolahSuhu` bernama `pengolah` dengan memberikan array tersebut ke constructor.

Program kemudian menampilkan data awal melalui `tampilkanData()`, sehingga Hari 4 muncul sebagai "(kosong)". Setelah itu, index hari kosong dicari dengan `cariIndexKosong()` dan hasilnya (3) dicetak.

Langkah berikutnya adalah menjalankan `isiDataKosong()` untuk mengisi data yang kosong. Data yang sudah bersih ditampilkan kembali melalui `tampilkanData()`, lalu rata-rata suhu dihitung dengan `hitungRataRata()` dan dibulatkan menjadi dua angka di belakang koma, sehingga hasilnya `29.39°C`.

Terakhir, isi array `suhuHarian` dicetak langsung dari `main`, bukan melalui object. Pencetakan dilakukan dengan perulangan `for`. Tanda koma dan spasi hanya dicetak untuk elemen yang bukan elemen terakhir, sehingga format yang dihasilkan menjadi daftar yang diapit tanda kurung siku.

10. Penjelasan Perubahan Array di `main`

Array `suhuHarian` di `main` ikut berubah menjadi `[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]`, padahal `main` tidak pernah mengubahnya secara langsung. Hal ini terjadi karena array di Java bertipe `reference`. Variabel `suhuHarian` di `main` hanya menyimpan alamat array di memori, bukan isi array itu sendiri. Pada saat constructor dijalankan, yang disalin ke field adalah alamat tersebut, bukan isi arraynya. Dengan demikian, field di dalam object dan variabel di `main` menunjuk ke satu array yang sama di memori. Ketika `isiDataKosong()` mengubah elemen array melalui object, perubahan tersebut otomatis terlihat dari `main` karena keduanya mengakses array yang sama.

D. Kesimpulan

Berdasarkan praktikum Modul 3 tentang Konsep Dasar Pemrograman Berorientasi Objek, dapat disimpulkan bahwa OOP membangun program dari objek yang menggabungkan data

(atribut) dan perilaku (method) dalam satu kesatuan. Class berperan sebagai blue print, sedangkan object dibuat dari class tersebut menggunakan keyword `'new'`, dan setiap object dapat memiliki nilai atribut yang berbeda. Pada bagian guided, program Buku, Lagu, dan Circle menunjukkan cara mendefinisikan class beserta field dan method, membuat object, serta memanggil method dari object tersebut. Program Buku dan Lagu juga menunjukkan penggunaan keyword `'this'` untuk membedakan field dari parameter yang namanya sama, dan untuk memanggil constructor lain dari dalam sebuah constructor.

Praktikum ini juga menunjukkan perbedaan class field dan instance field. Class field menggunakan modifier `'static'`, dimiliki oleh class, dan dipakai bersama, seperti konstanta PI pada class Circle. Instance field dimiliki oleh masing-masing object, seperti jari-jari lingkaran atau judul buku. Pada percobaan passing parameter, tipe data primitif dilewatkan secara pass by value sehingga perubahan di dalam method tidak memengaruhi variabel asal, sedangkan object dan array dilewatkan melalui referensi sehingga perubahan di dalam method juga terlihat pada data asalnya. Percobaan abstraksi dan enkapsulasi menunjukkan bahwa abstraksi memilih atribut dan perilaku yang penting dari suatu objek, sedangkan enkapsulasi melindungi data dengan akses `'private'` dan menyediakan method sebagai satu-satunya jalur untuk mengakses atau mengubah data tersebut, seperti pada class Rekening. Pada bagian unguided, konsep-konsep tersebut diterapkan untuk membangun class PengolahSuhu yang membersihkan dan menganalisis data suhu mingguan. Nilai penanda data kosong disimpan satu kali sebagai class field `'private static final'`, sedangkan data suhu disimpan sebagai instance field `'private'` agar terlindungi dari akses langsung di luar class. Method `'cariIndexKosong()'`, `'isiDataKosong()'`, dan `'hitungRataRata()'` berhasil menemukan data kosong pada index 3, mengisinya dengan rata-rata hari sebelum dan sesudahnya (imputasi) sehingga bernilai 29.1°C, lalu menghasilkan rata-rata suhu sebesar 29.39°C. Program ini juga membuktikan bahwa array bertipe reference: karena constructor hanya menyimpan alamat array, perubahan yang dilakukan melalui object ikut terlihat pada array di `'main'`.